How BiLSTM is Used in Sentiment Analysis

Name: Sharranyan Wijendran ID: 24180169 Github: Bidirectional-LSTM-2418016

Word count: ~1,800 words (excludes references, tables, figure captions, and code blocks)

Parameter	Value
Technique	Bidirectional LSTM (BiLSTM)
Task	3-class sentiment: Positive / Neutral / Negative
Dataset	Amazon iPhone Reviews (3,062 reviews)
Test accuracy	83.56% on held-out reviews

Accessibility: This document uses a color-blind friendly palette (blue and orange, not red and green). All figures include plain-text descriptions. Heading styles are used throughout for screen reader navigation.

1. Introduction

The process of classifying text as good, negative, or neutral is known as sentiment analysis. It is one of the most widely used applications of natural language processing, and companies utilize it to understand consumer sentiment at a scale that is impossible for a human team to handle by hand.

In order to categorize Amazon iPhone reviews as Positive, Neutral, or Negative, this tutorial develops a Bidirectional LSTM (BiLSTM) model. It demonstrates a thorough implementation on actual data after covering the entire theory, including the mathematics of the gates.

2. Why BiLSTM for Text?

Text is sequential and word order changes meaning completely:

Sentence	Sentiment
"I like this phone"	Positive
"I do NOT like this phone"	Negative

Table 1: Sample Sentiment Classification Results

There are very similar terms in both sentences. The negation cannot be identified by a model that ignores word order. BiLSTM processes text in both ways simultaneously and analyses it sequentially, providing each word with the whole context of the entire sentence.

Model	Reads	Weakness	Accuracy (est.)
SimpleRNN	Left → right	Forgets early words (vanishing gradient)	65–72%
LSTM	Left → right	No backward context	74–80%
BiLSTM	Both directions	Higher compute cost	78–86%

Table 2: Comparison of RNN, LSTM, and BiLSTM Models

3. From SimpleRNN to BiLSTM

3.1 SimpleRNN and the Vanishing Gradient Problem

The simplest RNN reads one word at a time. At each step t it produces a hidden state h(t) from the current word x(t) and the previous hidden state h(t-1). This is the model's running memory:

$$h(t) = tanh(W \cdot x(t) + U \cdot h(t-1) + b)$$

x(t) = current word vector
h(t-1) = previous hidden state
W, U = learned weight matrices
b = bias term
tanh = activation function

The issue comes up during training. In order to train, the model propagates error gradients backward via each word step. The gradient is multiplied by the weight matrix W at every stage. The gradient decreases at each step if the highest eigenvalue of W is less than 1, which is frequently the case. This equates to 100 multiplications for a 100-word review. The model stops learning from early words as the gradient gets closer to zero. According to Hochreiter and Schmidhuber, this is the vanishing gradient issue (Hochreiter and Schmidhuber, 1997).

Consider it a 100-person telephone chain. Everybody recalls the message a little differently. The original is lost by person 100. The issue with SimpleRNN is that the output step barely affects the word NOT at position t=3.

Figure 1 shows SimpleRNN processing "I do NOT like this phone" step by step:

"I"	"do"	**"NOT"**	"like"
RNN Cell t = 1	**RNN Cell** t = 2	**RNN Cell** t = 3	**RNN Cell** t = 4
h(1) → h(2) → h(3) → h(4) → Output			
Gradient signal: Strong at t=3 (NOT) → Shrinks at t=4 → Nearly zero at output			

Figure 1: Gradient Flow Through Time in a Simple RNN

3.2 LSTM - Solving Forgetting with Gates

By including a cell state C(t), a protected memory channel, and three learnable gates that regulate what is retained, discarded, and transmitted forward, (Hochreiter and Schmidhuber, 1997) LSTM addresses the vanishing gradient issue.

Activation Functions

Two activation functions are used inside LSTM. Understanding them first makes the gate equations much clearer:

Sigmoid σ(x)	**Tanh tanh(x)**
Output range: 0 to 1	Output range: -1 to +1
Acts like a dimmer switch	*Captures negative and positive signal*
0 = block completely	-1 = strongly negative
1 = pass through fully	+1 = strongly positive

Used in: all 3 gates	Used in: cell state, output

Figure 2: Activation Functions Used in LSTM Gates

The Three Gates and Their Equations

Every gate generates a value between 0 and 1 using a sigmoid. After then, element-wise multiplication (⊙) is used to scale the information flow by these values. The whole LSTM cell is seen in Figure 3

Cell State C(t) - protected memory highway ⟶

f(t) Forget Gate	i(t) Input Gate	o(t) Output Gate
$\sigma(Wf \cdot [h(t-1), x(t)] + bf)$	$\sigma(Wi \cdot [h(t-1), x(t)] + bi)$	$\sigma(Wo \cdot [h(t-1), x(t)] + bo)$
What should I forget?	*What should I store?*	*What do I pass forward?*

Cell state update:

$$C(t) = f(t) \odot C(t-1) + i(t) \odot tanh(Wc \cdot [h(t-1), x(t)] + bc)$$

Output hidden state:

$$h(t) = o(t) \odot tanh(C(t))$$

Inputs: x(t) current word + h(t-1) previous hidden state
Output: h(t) → next step + classifier

Figure 3: LSTM Cell Structure and Information Flow

Forget Gate - what to discard from memory

$$\boldsymbol{f(t) = \sigma(Wf \cdot [h(t-1), x(t)] + bf)}$$

f(t) → 0 means: forget this (discard from cell state)
f(t) → 1 means: keep this (preserve in cell state)

Example: when the model reads NOT, the forget gate learns to suppress previously stored positive sentiment signals.

Input Gate - what new information to store

$$\boldsymbol{i(t) = \sigma(Wi \cdot [h(t-1), x(t)] + bi)}$$ <- how much to write

$$\boldsymbol{C\sim(t) = tanh(Wc \cdot [h(t-1), x(t)] + bc)}$$ <- candidate values

Cell state is then updated:

$$\boldsymbol{C(t) = f(t) \odot C(t-1) + i(t) \odot C\sim(t)}$$

⊙ = element-wise multiplication
First term: how much old memory to keep
Second term: how much new information to add

Output Gate - what to pass forward

$$\boldsymbol{o(t) = \sigma(Wo \cdot [h(t-1), x(t)] + bo)}$$

$$\boldsymbol{h(t) = o(t) \odot tanh(C(t))}$$

tanh squishes the cell state to -1 to +1
output gate selects which parts to expose as h(t)
h(t) is passed to the next step and to the classifier

The cell state solves vanishing gradients because gradients can flow backwards via the addition without being multiplied away due to the additive nature of the update: C(t) = ... + i(t) ⊙ C̃(t). The gates are entirely learned: the model independently determines which words are significant enough to retain throughout the whole sequence.

3.3 BiLSTM - Reading Both Directions

A simple LSTM only reads from left to right. It has only seen I, do, NOT when it processes something like "I do NOT like this phone"; it is unsure of what comes next. (Schuster and Paliwal, 1997) invented the Bidirectional LSTM (BiLSTM), which runs two LSTM layers concurrently, one forward and one backward, and concatenates their outputs:

$$\boldsymbol{Forward}:\ \boldsymbol{h_fwd(t)} = \boldsymbol{LSTM_forward(x(t), h_fwd(t-1))}$$
$$\boldsymbol{Backward}:\ \boldsymbol{h_bwd(t)} = \boldsymbol{LSTM_backward(x(t), h_bwd(t+1))}$$
$$\boldsymbol{Combined}:\ \boldsymbol{h_hat(t)} = [\,\boldsymbol{h_fwd(t)};\ \boldsymbol{h_bwd(t)}]$$

[;] = concatenation
Two 64-unit LSTMs produce one 128-dimensional vector per word
Every word gets full context from both directions

Figure 4 shows the BiLSTM processing "I do NOT like this phone":

"I"	"do"	**"NOT"**	"like"	"this"	"phone"
LSTM → f1	**LSTM →** f2	**LSTM →** f3	**LSTM →** f4	**LSTM →** f5	**LSTM →** f6
← LSTM b1	**← LSTM** b2	**← LSTM** b3	**← LSTM** b4	**← LSTM** b5	**← LSTM** b6
concat	concat	concat	concat	concat	concat

Figure 4: Bidirectional LSTM Processing of a Sentence

What BiLSTM sees for the word NOT at position t=3:
Forward h_fwd(3): has seen I → do → NOT (negation is in memory)
Backward h_bwd(3): has seen phone → this → like → NOT (knows what NOT affects)
Combined h_hat(3) = [h_fwd(3) ; h_bwd(3)] → correctly predicts Negative ✓

3.4 End-to-End Computation Summary

Putting it all together, a review flows through these steps from raw text to prediction:

Step	Operation	What happens
1	Embedding	Word index w → vector e = E[w] (E = embedding matrix)
2	BiLSTM forward	h_fwd(1..N) computed left to right using LSTM equations
3	BiLSTM backward	h_bwd(1..N) computed right to left
4	Concatenate	h_hat = [h_fwd(N); h_bwd(1)] - final states of both directions
5	Dense (ReLU)	z = ReLU(W1·h_hat + b1) - higher-level feature learning
6	Output (Softmax)	y_hat = softmax(W2·z + b2) - class probabilities

7	Prediction	class = argmax(y_hat) - highest probability wins

Table 3: Step-by-Step Workflow of a BiLSTM Neural Network

3.5 Training - Loss Function

The model minimizes the categorical cross-entropy loss, which calculates the deviation between the true labels and predicted probabilities:

$$L = -SUM\ y_c * \log\left(y_{hat_c}\right)$$

c in {Negative, Neutral, Positive}

Example: true label = Positive (y = [0, 0, 1])
model predicts y_hat = [0.05, 0.10, 0.85]
L = -(0*log (0.05) + 0*log (0.10) + 1*log (0.85)) = 0.163

Adam optimiser with learning rate 0.0005 updates weights
after each batch to minimise L.

4. Model Architecture

There are seven layers in the model. Sentiment probabilities are derived from raw word indexes:

Layer	Output shape	What it does
Input	(200,)	200 integer word indices per review
Embedding (5000 × 128)	(200, 128)	Each word index → 128-dim semantic vector
Bidirectional LSTM (64)	(128,)	64 forward + 64 backward = 128 total features
Dropout 40%	(128,)	Randomly disables neurons to prevent overfitting
Dense 32 (ReLU)	(32,)	Higher-level feature combinations
Dropout 30%	(32,)	Additional regularization
Dense 3 (Softmax)	(3,)	Outputs probability for Negative, Neutral, Positive

Table 4: Layer-wise Architecture of the BiLSTM Sentiment Classification Model

Compiled with Adam (learning rate 0.0005) and categorical cross-entropy loss. Early stopping with patience=7 halts training when validation loss stops improving and restores the best weights automatically.

5. Dataset and Preprocessing

5.1 Dataset

At first, the dataset contains 3,062 Amazon iPhone reviews from Amazon India. The star ratings were subsequently mapped to sentiment labels:

Rating	Label	Count	Class weight
1–2 stars	Negative	~758 (25%)	1.33
3 stars	Neutral	~239 (8%)	4.22
4–5 stars	Positive	~2065 (67%)	0.50

Table 5: Class Distribution and Weights for Sentiment Labels

5.2 Label Noise Removal

The final dataset consisted of 2,979 reviews after 83 conflicting reviews (2.7%) were removed due to noise. Reviews with inconsistent ratings and content are removed using TextBlob polarity scores. (Steven Loria, 2018):

Case	Rating	Text polarity	Action
Case A	1–2	Above +0.3	Remove - positive text, low rating
Case B	4–5	Below −0.3	Remove - negative text, high rating

Table 6: Criteria for Removing Inconsistent Sentiment Samples

5.3 Preprocessing Pipeline

Step	Operation	Example
1	Combine title + description	"No charger Everything is good but..."
2	Lowercase	"No Charger" → "no charger"
3	Remove URLs and symbols	"price=62k!!" → "price k"
4	Tokenize: words → integers	"no charger" → [34, 128]
5	Pad to length 200	[34, 128, 0, 0, ..., 0]

Table 7: Text Pre-processing Pipeline for Sentiment Analysis

6. Results

6.1 Training

Trained to stop early (patience = 7). Weights from epoch 3 were recovered after stopping at epoch 10. This is the estimated action; the model started overfitting at epoch 3 and generalized the best after that.

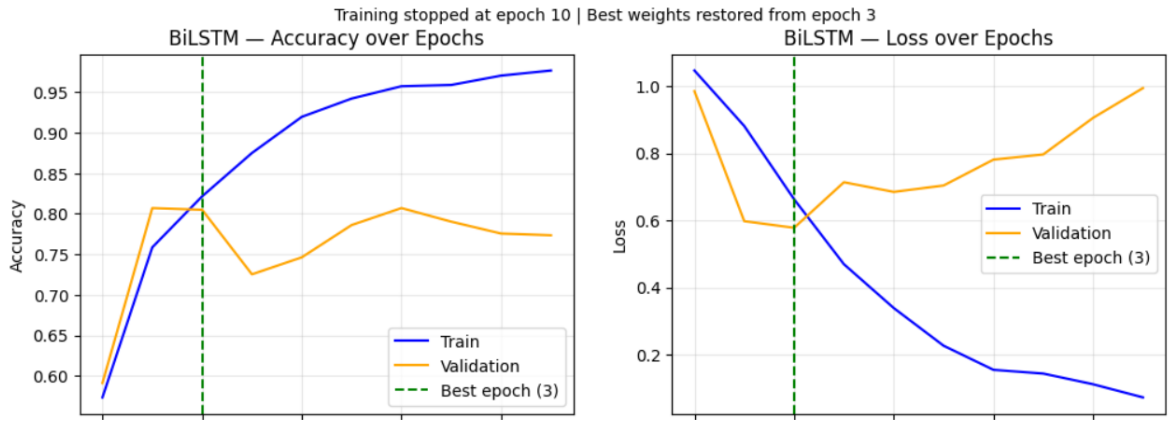

Figure 5: Training vs Validation Loss & Accuracy Curve

Early stopping with restoring weight was successful, as seen by the training curve above Figure 5, which demonstrates that validation loss was lowest at epoch 3. The two Dropout layers somewhat reduce the slight overfitting shown by the difference between training and validation loss after epoch 3.

6.2 Test Set Performance

596 held-out reviews (20%). Overall accuracy: 83.6%. Macro F1: 0.65.

Class	Precision	Recall	F1	Test reviews
Negative	0.73	0.88	0.80	144
Neutral	0.23	0.22	0.22	46
Positive	0.96	0.89	0.92	406
Macro avg	0.64	0.66	0.65	596

Table 8: Precision, Recall, and F1-Score Across Sentiment Classes

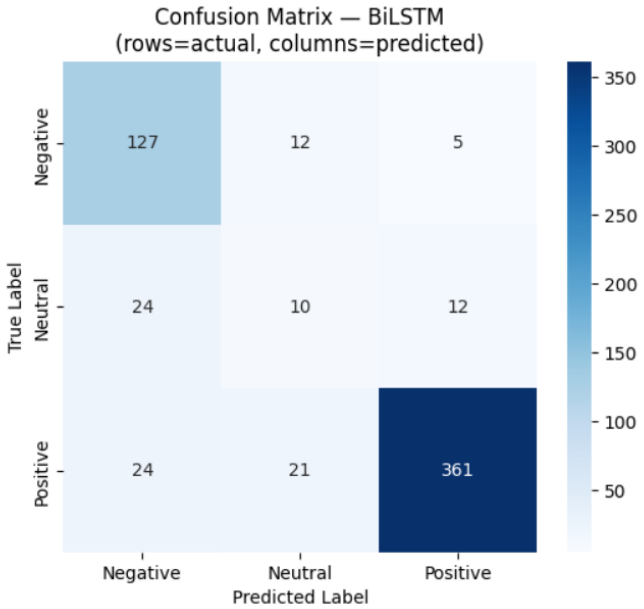

Figure 6: Confusion Matrix

The method performs best on positive reviews, properly classifying 89% of them, according to the confusion matrix shown in Figure 6. At 88% recall, negative evaluations are easily recognised. With just 46 neutral test samples and overlapping terminology, the Neutral class is the weakest, with many evaluations incorrectly labelled as Positive or Negative.

6.3 Discussion

- **Positive (F1 = 0.92):** Strong - enough data, consistent vocabulary.
- **Negative (F1 = 0.80):** Good - struggles with mild or mixed language.
- **Neutral (F1 = 0.22):** Weak - only 46 training examples. Neutral language overlaps with both classes.

Accuracy range: 92–94% is the practical limit for this task. On reviews that are on the edge of three stars, even human reviewers cannot agree. The model is learning genuine signal, as shown by reaching 83.6%, which is 11.7 points higher than the majority-class baseline of 67%.

7. Conclusion

This tutorial followed an entire development arc from the vanishing gradient problem of SimpleRNN to the gated memory of LSTM to the bidirectional context of BiLSTM. Bidirectional context really enhances sentiment classification, as shown by the BiLSTM's 83.6% test accuracy, which is 11.7 points higher than the majority-class baseline when applied to 2,979 Amazon iPhone reviews. Due to overlapping language and a lack of training examples, the neutral class (F1 = 0.22) is the primary limitation. Future research could look into BERT fine-tuning for cutting-edge outcomes or GloVe embeddings for fuller word representations.

References

Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. [online] ArXiv. Available at: https://arxiv.org/abs/1810.04805.

Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, [online] 9(8), pp.1735–1780. doi:https://doi.org/10.1162/neco.1997.9.8.1735.

Steven Loria (2018). *TextBlob: Simplified Text Processing — TextBlob 0.15.2 documentation*. [online] Readthedocs.io. Available at: https://textblob.readthedocs.io/en/dev/.

Pennington, J., Socher, R. and Manning, C.D. (2014). *GloVe: Global Vectors for Word Representation*. [online] Stanford.edu. Available at: https://nlp.stanford.edu/projects/glove/.

Zhang, L., Wang, S. and Liu, B. (2018). Deep learning for sentiment analysis: A survey. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, [online] 8(4). doi:https://doi.org/10.1002/widm.1253.

GeeksforGeeks (2023). *Bidirectional LSTM in NLP*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/nlp/bidirectional-lstm-in-nlp/.

malak, A. (2023). *RNN & LSTM & BI-LSTM & GRU*. [online] Medium. Available at: https://medium.com/@antoniosmalak14/rnn-lstm-bi-lstm-gru-eb5869660691.

contributors, M.C. and (n.d.). *Basic Syntax | Markdown Guide*. [online] www.markdownguide.org. Available at: https://www.markdownguide.org/basic-syntax/.

TensorFlow. (n.d.). tf.keras.Layer | TensorFlow v2.16.1. [online] Available at: https://www.tensorflow.org/api_docs/python/tf/keras/Layer.

Schuster, M. and Paliwal, K.K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45(11), pp.2673–2681. doi:https://doi.org/10.1109/78.650093.